Artificial Intelligence Application Development Practical

Smart Book Recommendation System Using Machine Learning

Student Name: Neeraj Mishra

Roll Number: T021

Class: B.Sc. Information Technology

Division: A

Subject: Artificial Intelligence Application Development Practical

Academic Year: 2026–2027

College Name: SHETH L.U.J & SIR MV COLLEGE

Department Name: IT

Faculty Name: Ms. Chetana Tanawade

1. INTRODUCTION

The Smart Book Recommendation System Using Machine Learning is an intelligent application designed to recommend books to readers based on their interests and preferences. With a very large number of books available in online stores and digital libraries, readers often find it difficult to decide what to read next. A recommendation system solves this problem by identifying books that are similar to the book selected by the user.

This project uses Machine Learning techniques such as Content-Based Filtering, TF-IDF Vectorization, and Cosine Similarity to analyze book information such as title, genre, and description. The system converts book information into numerical features and calculates the similarity between books. Based on the similarity scores, the system recommends the most relevant books.

The application is developed using Python, Pandas, Scikit-learn, and Streamlit. It provides a simple interface where users can select a book and receive a list of similar book recommendations along with their similarity scores.

The main purpose of this project is to demonstrate how Machine Learning can be used to provide personalized and intelligent book recommendations in an easy-to-use application.

2. PROBLEM STATEMENT

With the increasing number of books available in online bookstores and digital libraries, readers often find it difficult to choose a book that matches their interests. Searching through a large collection of books can be time-consuming and may not always provide relevant results.

The problem is to develop a Smart Book Recommendation System that can analyze book information such as genre and description and recommend books that are similar to the book selected by the user.

The system uses Machine Learning techniques such as Content-Based Filtering, TF-IDF Vectorization, and Cosine Similarity to calculate the similarity between books and generate relevant recommendations. The system should provide the recommendations through a simple and user-friendly interface.

3. OBJECTIVES

The main objectives of the Smart Book Recommendation System Using Machine Learning are:

1 To develop a smart recommendation system that suggests books based on the user's selected book.

2 To analyze book information such as title, genre, and description.

3 To implement Content-Based Filtering for generating relevant book recommendations.

4 To use TF-IDF Vectorization to convert book information into numerical features.

5 To calculate book similarity using the Cosine Similarity technique.

6 To recommend the most similar books based on their similarity scores.

7 To display the recommendations through a simple and user-friendly Streamlit interface.

8 To demonstrate the practical application of Machine Learning in developing an intelligent book recommendation system.

4. TECHNOLOGIES USED

The following technologies and tools are used to develop the Smart Book Recommendation System Using Machine Learning:

1 **Python** – Used as the main programming language for developing the recommendation system.

2 **Pandas** – Used for creating, storing, and processing the book dataset.

3 **Scikit-learn** – Used to implement Machine Learning techniques such as TF-IDF Vectorization and Cosine Similarity.

4 **TF-IDF Vectorization** – Converts book genres and descriptions into numerical feature vectors.

5 **Cosine Similarity** – Measures the similarity between books and helps identify the most relevant recommendations.

6 **Streamlit** – Used to create the interactive and user-friendly web interface for selecting books and displaying recommendations.

7 **VS Code** – Used as the development environment for writing and running the Python application.

5. DATASET

The Smart Book Recommendation System uses a sample book dataset containing information about different books. The dataset is created specifically for this project and is stored as a Pandas DataFrame in the Python program.

Dataset Attributes:

Attribute	Description
Title	Name of the book
Genre	Genre or category of the book, such as Fantasy, Mystery, Science Fiction, etc.
Description	Short description containing important keywords about the book and its content

The dataset contains 20 sample books, including popular fantasy, science fiction, and mystery titles. The Genre and Description fields are combined to create a Features column.

These features are then processed using TF-IDF Vectorization to convert the text into numerical values. Cosine Similarity is applied to these values to determine how similar one book is to another.

Dataset Processing Flow

Book Dataset → Data Cleaning → Feature Combination → TF-IDF Vectorization → Cosine Similarity → Book Recommendations

The dataset is used for educational purposes to demonstrate how a Machine Learning-based book recommendation system works.

6. METHODOLOGY / WORKING

The Smart Book Recommendation System works by analyzing book information and finding books that are similar to the book selected by the user. The complete methodology is as follows:

Step 1: Data Collection

A sample book dataset is created containing Title, Genre, and Description of 20 books.

Step 2: Data Preparation

The book data is stored in a Pandas DataFrame. The Genre and Description of each book are combined to create a single Features column.

Step 3: TF-IDF Vectorization

The TF-IDF (Term Frequency–Inverse Document Frequency) technique converts the text in the Features column into numerical vectors. This allows the system to compare the important words present in different books.

Step 4: Calculate Cosine Similarity

Cosine Similarity is applied to the TF-IDF vectors to measure the similarity between every pair of books. A higher similarity score indicates that two books have more similar content.

Step 5: Select a Book

The user selects a book from the Streamlit interface using a dropdown menu.

Step 6: Generate Recommendations

The system finds the selected book in the similarity matrix and compares its similarity scores with other books. The books with the highest similarity scores are selected as recommendations.

Step 7: Display Results

The recommended books are displayed along with their genre and similarity percentage. A similarity chart can also be displayed to provide a visual comparison.

Overall Working

Book Dataset → Feature Combination → TF-IDF Vectorization → Cosine Similarity → Select Book → Find Similar Books → Display Recommendations

This methodology allows the system to provide content-based book recommendations without requiring information about other users.

7. ALGORITHM USED

The Smart Book Recommendation System uses Content-Based Filtering with TF-IDF Vectorization and Cosine Similarity to recommend books.

1. Content-Based Filtering

Content-Based Filtering recommends books based on the features and content of the books. In this project, the system uses the Genre and Description of books to identify books with similar content.

2. TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) converts the text information of books into numerical vectors.

- It identifies important words in the book descriptions and genres.
- Common words are given less importance.
- Important words receive higher weights.
- These numerical vectors are used for comparing books.

3. Cosine Similarity

Cosine Similarity measures the similarity between the TF-IDF vectors of two books.

- A higher similarity score means the books have more similar content.
- The system calculates similarity between the selected book and all other books.
- Books with the highest similarity scores are recommended.

Algorithm Steps

1 Store the book dataset.

2 Combine Genre and Description into a Features column.

3 Apply TF-IDF Vectorization to the Features column.

4 Calculate Cosine Similarity between all books.

5 Allow the user to select a book.

6 Find the selected book's similarity scores.

7 Sort the books according to their similarity scores.

8 Select the top similar books.

9 Display the recommended books with their similarity scores.

Algorithm Flow:

Book Data → Feature Extraction → TF-IDF → Cosine Similarity → Similarity Ranking → Top Books → Recommendations

8. IMPLEMENTATION

The Smart Book Recommendation System is implemented using Python, Pandas, Scikit-learn, and Streamlit. The system uses Content-Based Filtering, TF-IDF Vectorization, and Cosine Similarity to generate book recommendations.

CODE:

```
import streamlit as st
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# ================================================
# PAGE CONFIGURATION
# ================================================

st.set_page_config(
    page_title="AI Book Recommendation System",
    page_icon="",
    layout="wide"
)

# ================================================
# TITLE
# ================================================

st.title("AI-Based Book Recommendation System")
st.write(
```

```
    "An AI-based book recommendation system that recommends "
    "similar books using Content-Based Filtering, TF-IDF "
    "and Cosine Similarity."
)

st.write("A Project Made By Neeraj Mishra (T021)")

# ================================================
# SAMPLE BOOK DATASET
# ================================================

data = {

    "Title": [
        "Harry Potter and the Sorcerers Stone",
        "Harry Potter and the Chamber of Secrets",
        "Harry Potter and the Prisoner of Azkaban",
        "The Hobbit",
        "The Fellowship of the Ring",
        "The Two Towers",
        "The Lion the Witch and the Wardrobe",
        "Percy Jackson The Lightning Thief",
        "The Hunger Games",
        "Catching Fire",
        "Divergent",
        "The Maze Runner",
        "Dune",
        "Enders Game",
        "The Da Vinci Code",
        "Angels and Demons",
        "A Study in Scarlet",
        "Murder on the Orient Express",
        "Gone Girl",
        "The Girl with the Dragon Tattoo"
    ],

    "Genre": [
        "Fantasy Adventure",
        "Fantasy Adventure Mystery",
        "Fantasy Adventure Mystery",
        "Fantasy Adventure",
        "Fantasy Adventure",
        "Fantasy Adventure",
        "Fantasy Adventure Children",
        "Fantasy Adventure Mythology",
        "Science Fiction Dystopian Thriller",
        "Science Fiction Dystopian Thriller",
        "Science Fiction Dystopian Thriller",
        "Science Fiction Dystopian Adventure",
        "Science Fiction Adventure",
        "Science Fiction Military",
        "Mystery Thriller",
        "Mystery Thriller",
        "Mystery Crime Detective",
        "Mystery Crime Detective",
        "Mystery Thriller Psychological",
        "Mystery Crime Thriller"
    ],

    "Description": [
        "harry potter young wizard magic school hogwarts friendship",
        "harry potter wizard magic school hogwarts secret chamber mystery",
        "harry potter wizard magic school hogwarts prisoner escape mystery",
        "hobbit bilbo dwarves dragon treasure journey middle earth",
        "frodo ring journey fellowship middle earth wizard quest",
        "frodo aragorn war middle earth quest battle fellowship",
        "children magical land narnia wardrobe lion witch adventure",
        "percy jackson greek gods demigod mythology quest camp",
        "katniss survival games arena rebellion dystopian capitol",
        "katniss survival games arena rebellion district capitol",
        "tris faction society future rebellion dystopian choice",
        "maze teenagers survival escape mystery dystopian glade",
        "desert planet spice politics future empire survival",
        "child genius space war strategy battle school future",
        "symbologist secret code church art conspiracy mystery",
        "symbologist vatican conspiracy secret society mystery",
        "sherlock holmes detective murder investigation london",
        "detective hercule poirot murder train investigation suspects",
        "missing wife marriage secrets psychological twist crime",
        "journalist hacker investigation missing girl crime secrets"
    ]
}
df = pd.DataFrame(data)
```

```
# ================================================
# CREATE COMBINED FEATURES
# ================================================

df["Features"] = (
    df["Genre"] + " " +
    df["Description"]
)

# ================================================
# TF-IDF VECTORIZATION
# ================================================

vectorizer = TfidfVectorizer(
    stop_words="english"
)

feature_matrix = vectorizer.fit_transform(
    df["Features"]
)

# ================================================
# COSINE SIMILARITY
# ================================================

similarity_matrix = cosine_similarity(
    feature_matrix
)

# ================================================
# RECOMMENDATION FUNCTION
# ================================================

def recommend_books(book_name, number_of_books=5):

    book_index = df[
        df["Title"] == book_name
    ].index[0]

    similarity_scores = list(
        enumerate(similarity_matrix[book_index])
    )
    similarity_scores = sorted(
        similarity_scores,
        key=lambda x: x[1],
        reverse=True
    )

    recommendations = []

    for index, score in similarity_scores[1:number_of_books + 1]:

        recommendations.append({
            "Book": df.iloc[index]["Title"],
            "Genre": df.iloc[index]["Genre"],
            "Similarity": score
        })

    return pd.DataFrame(recommendations)

# ================================================
# USER INPUT
# ================================================

st.header("Select a Book")

selected_book = st.selectbox(
    "Choose a book you like:",
    df["Title"]
)

number_of_books = st.slider(
    "Number of recommendations",
    min_value=3,
    max_value=10,
    value=5
)

# ================================================
# RECOMMENDATION BUTTON
# ================================================

if st.button("Recommend Books"):
```

```
    recommendations = recommend_books(
        selected_book,
        number_of_books
    )

    st.subheader("Recommended Books")

    for i, row in recommendations.iterrows():
        st.write(f"### {i + 1}. {row['Book']}")
        st.write(f"**Genre:** {row['Genre']}")
        st.write(
            f"**Similarity Score:** "
            f"{row['Similarity'] * 100:.2f}%"
        )
        st.progress(float(row["Similarity"]))
        st.divider()

# ================================================
# RECOMMENDATION CHART
# ================================================

if st.button("Show Recommendation Chart"):

    recommendations = recommend_books(
        selected_book,
        number_of_books
    )

    chart_data = recommendations.set_index(
        "Book"
    )["Similarity"]

    st.subheader("Book Similarity Chart")
    st.bar_chart(chart_data)

# ================================================
# DATASET PREVIEW
# ================================================

st.header("Book Dataset")

with st.expander("View Book Dataset"):
    st.dataframe(
        df[["Title", "Genre", "Description"]],
        use_container_width=True
    )

# ================================================
# HOW THE SYSTEM WORKS
# ================================================

st.header("How AI Recommendation Works")

st.write(
    """
    1. The system stores book information such as title,
       genre and description.
    2. Book information is converted into numerical
       features using TF-IDF Vectorization.
    3. Cosine Similarity calculates how similar books
       are to each other.
    4. Books with the highest similarity scores are
       selected as recommendations.
    5. The recommended books are displayed through
       the Streamlit interface.
    """
)

# ================================================
# ABOUT PROJECT
# ================================================

st.header("About This Project")

st.write(
    """
    This project uses Content-Based Filtering to recommend
    books based on their similarity.
    TF-IDF is used to convert book descriptions and genres
    into numerical features, while Cosine Similarity is used
    to measure similarity between books.
    """
)
```

```
# ================================================
# DISCLAIMER
# ================================================

st.info(
    " This project is developed for educational purposes "
    "to demonstrate Artificial Intelligence and Machine "
    "Learning concepts."
)
```

CODE EXPLANATION:

The Smart Book Recommendation System is developed using Python, Pandas, Scikit-learn, and Streamlit. First, the required libraries are imported and a sample book dataset containing Title, Genre, and Description is created using Pandas. The Genre and Description are combined into a single Features column, which provides the information required for recommendation. The TF-IDF Vectorizer converts these textual features into numerical vectors, and Cosine Similarity calculates the similarity between all books. A recommendation function takes the book selected by the user, finds its similarity scores, sorts the books from highest to lowest similarity, and selects the top similar books while excluding the selected book itself. The Streamlit interface allows the user to select a book and choose the number of recommendations. When the Recommend Books button is clicked, the system displays the recommended books along with their genres, similarity scores, and progress bars. A similarity bar chart can also be displayed for comparison. The dataset can be viewed through the Streamlit interface using an expandable section. Thus, the complete code follows the process Book Dataset → Feature Creation → TF-IDF → Cosine Similarity → Similarity Ranking → Book Recommendations → Streamlit Output.

OUTPUT:

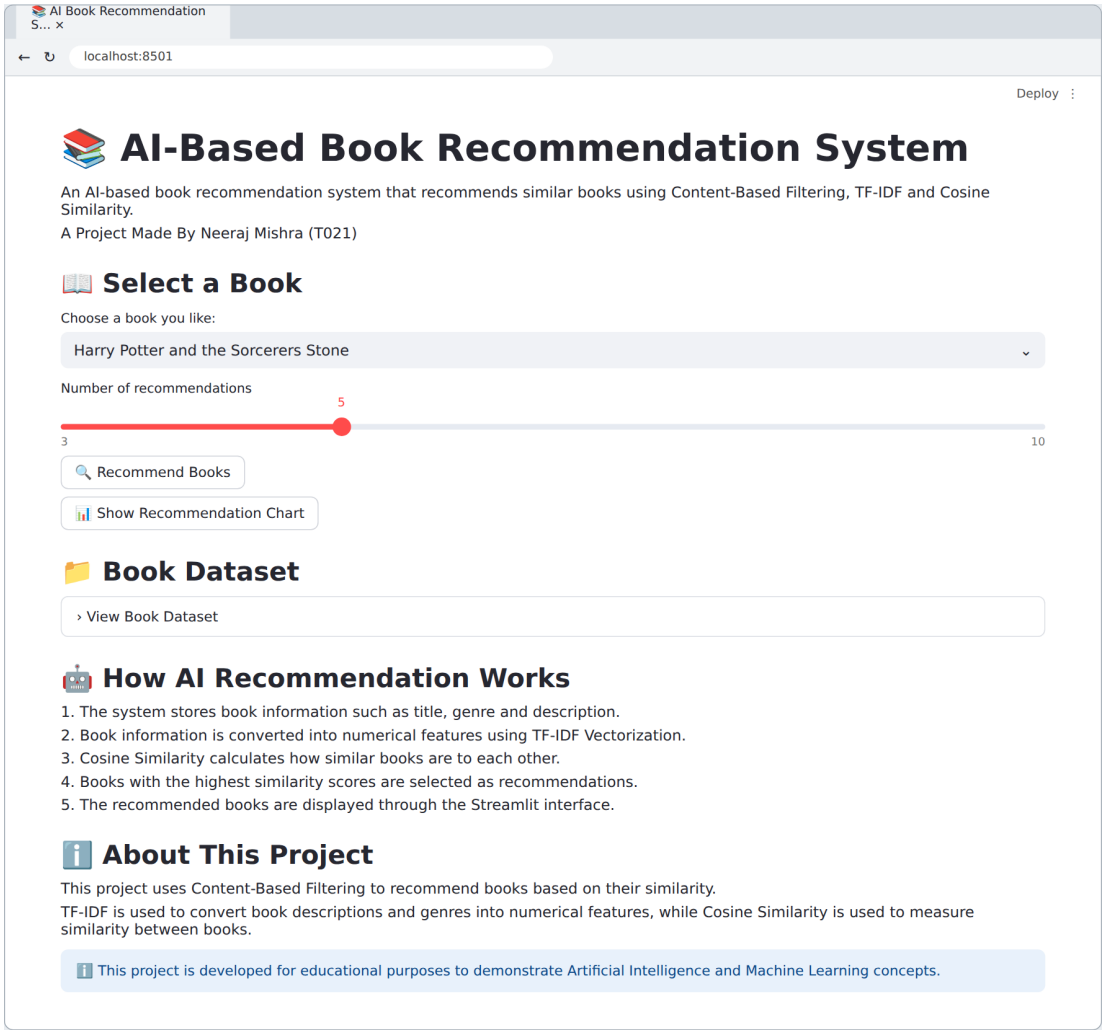


Figure 1: Home page – select a book and the number of recommendations

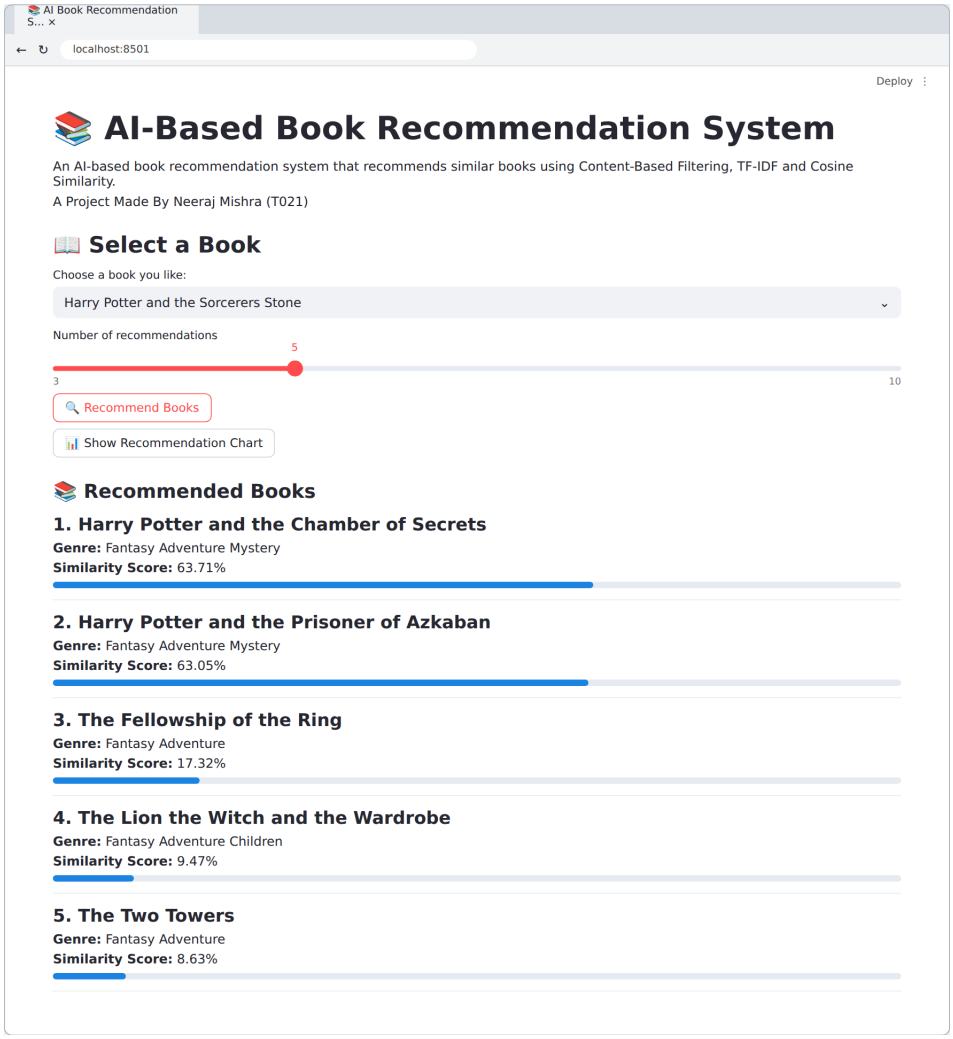


Figure 2: Recommended books with genre, similarity score and progress bars

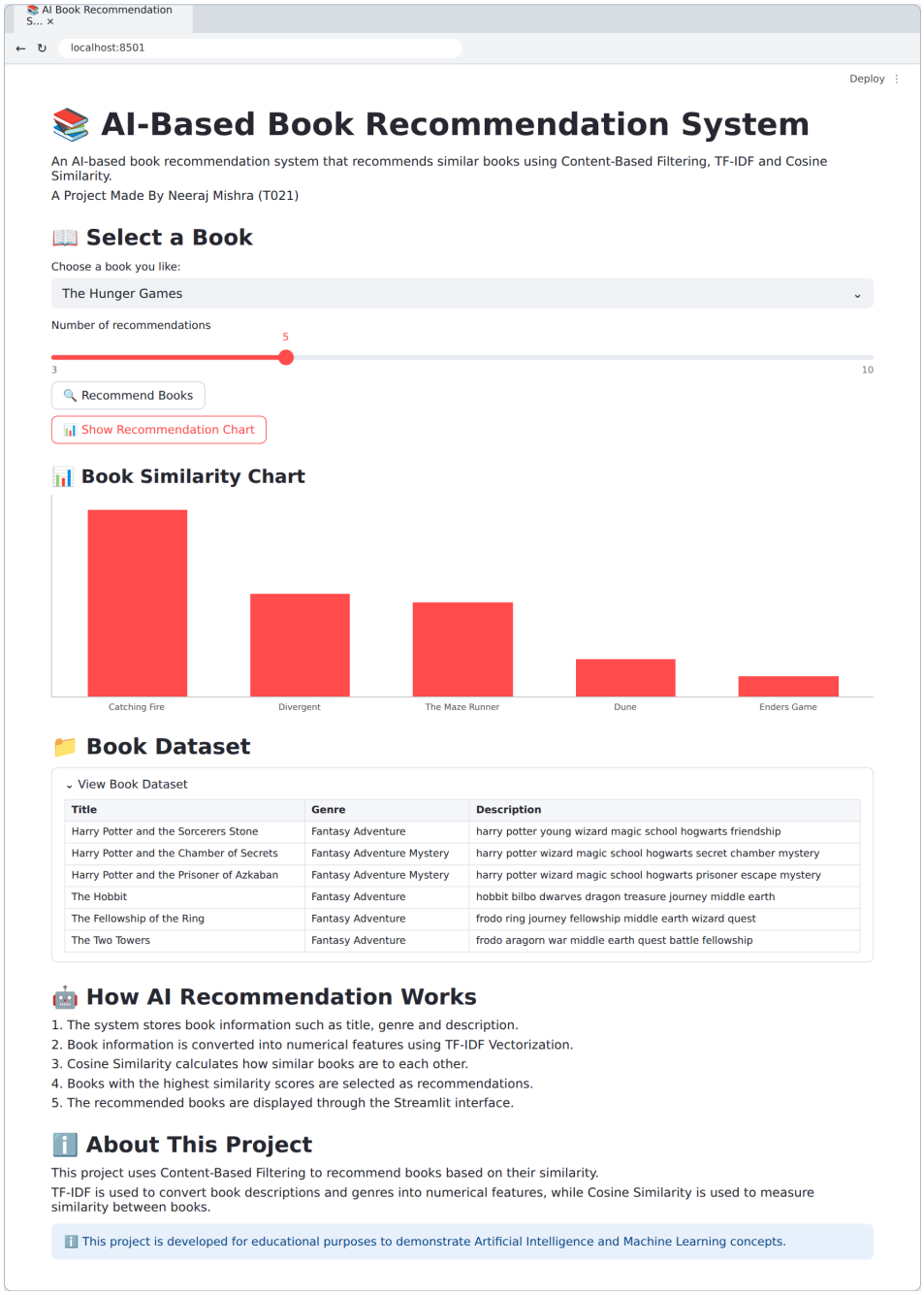


Figure 3: Book similarity chart and dataset preview

RESULT

The Smart Book Recommendation System Using Machine Learning was successfully developed and implemented using Python, Pandas, Scikit-learn, and Streamlit. The system analyzes book genres and descriptions using TF-IDF Vectorization and calculates similarity between books using Cosine Similarity.

When a user selects a book, the system successfully identifies and displays the most similar books along with their genre and similarity scores. The system also provides a book similarity chart for better visualization and allows users to view the complete book dataset.

The final result demonstrates that Content-Based Filtering can be effectively used to provide relevant book recommendations based on the content of a selected book. The

project successfully demonstrates the practical application of Machine Learning and AI concepts in recommendation systems.

ADVANTAGES

1. **Easy to Use** – The Streamlit interface provides a simple and user-friendly way to select a book and get recommendations.
2. **Relevant Recommendations** – The system recommends books based on the similarity of their genre and description.
3. **Saves Time** – Users can quickly find similar books without manually searching through a large collection.
4. **Content-Based Recommendations** – Recommendations are generated based on book content, so information about other users is not required.
5. **Uses Machine Learning Techniques** – TF-IDF and Cosine Similarity provide an effective method for comparing book content.
6. **Similarity Scores** – The system displays similarity scores, helping users understand how closely the recommended books match the selected book.
7. **Visual Representation** – The similarity chart provides an easy-to-understand comparison of recommended books.
8. **Simple and Low-Cost Implementation** – The system can be developed using freely available Python libraries without requiring an external recommendation API.

LIMITATIONS

1. **Limited Dataset** – The system uses a small sample dataset containing only 20 books, so recommendations may be limited.
2. **Limited Features** – Recommendations are based mainly on book genre and description and do not consider factors such as ratings, authors, or user preferences.
3. **No User History** – The system does not learn from the user's previous reading history or personal preferences.
4. **Content Dependency** – The quality of recommendations depends on the information available in the book descriptions and genres.
5. **No Real-Time Data** – The system does not use live book databases or continuously updated book information.
6. **Cold-Start Limitation** – New books with limited or incomplete information may not receive highly accurate recommendations.
7. **No Collaborative Filtering** – The system does not compare the user's preferences with the preferences of other users.
8. **Educational Prototype** – The system is mainly developed to demonstrate Machine Learning and recommendation-system concepts and may require a larger, real-world dataset for practical deployment.

APPLICATIONS

The Smart Book Recommendation System Using Machine Learning can be used in the following applications:

- **Online Bookstores** – Can recommend books similar to the book viewed or purchased by the user.
- **Digital Libraries** – Helps readers discover books that match their interests.
- **E-Book Platforms** – Provides customized book suggestions based on book content and user selection.
- **Book Search Systems** – Helps users find similar books quickly from a large collection.
- **Reading Applications** – Can be integrated into mobile or web-based reading applications.
- **College and School Libraries** – Can help students find related books and reference material.
- **Educational Projects** – Useful for demonstrating Machine Learning, Natural Language Processing, TF-IDF, and Cosine Similarity concepts.
- **Future Recommendation Systems** – Can be extended with ratings, user history, and collaborative filtering to build more advanced recommendation platforms.

CONCLUSION

The Smart Book Recommendation System Using Machine Learning was successfully developed using Python, Pandas, Scikit-learn, and Streamlit. The system uses Content-Based Filtering to recommend books based on their genre and description.

TF-IDF Vectorization is used to convert book information into numerical features, while Cosine Similarity is used to measure the similarity between books. Based on the similarity scores, the system successfully identifies and recommends the most similar books to the one selected by the user.

The Streamlit interface makes the system simple and user-friendly by allowing users to select a book, choose the number of recommendations, and view the recommended books with their similarity scores and charts.

Overall, the project demonstrates the practical application of Machine Learning, Natural Language Processing, and recommendation techniques to create an intelligent book recommendation system.

FUTURE SCOPES

The Smart Book Recommendation System Using Machine Learning can be further improved and expanded in the following ways:

1. **Larger Book Dataset** – Use a larger and regularly updated dataset containing thousands of books.
2. **User Preferences** – Allow users to provide their preferred genres, languages, or authors for more personalized recommendations.
3. **User Ratings and Reviews** – Include book ratings and reviews to improve the quality of recommendations.
4. **Collaborative Filtering** – Implement collaborative filtering to recommend books based on the preferences and reading patterns of similar users.
5. **Hybrid Recommendation System** – Combine content-based filtering and collaborative filtering to provide more accurate recommendations.

6 **User History** – Store previously read or liked books and use this information to generate personalized recommendations.

7 **Advanced Machine Learning** – Use advanced ML or deep learning techniques to improve recommendation accuracy.

8 **Real-Time Book Database** – Connect the system with an online book database or API to provide updated information about newly released books.

9 **Mobile Application** – Develop a mobile version so users can access book recommendations from smartphones.

10 **Improved User Interface** – Add book covers, previews, ratings, filters, and search features to make the system more interactive and attractive.

REFERENCES

1 Python Documentation – Python programming language and libraries.

2 Pandas Documentation – Used for creating and processing the book dataset.

3 Scikit-learn Documentation – Used for TF-IDF Vectorization and Cosine Similarity.

4 Streamlit Documentation – Used for developing the interactive web application.

5 Machine Learning and Recommendation System Study Materials – Used to understand Content-Based Filtering and recommendation techniques.

6 Project-Created Book Dataset – Sample dataset containing book titles, genres, and descriptions used for educational implementation.

GITHUB

https://github.com/neermishmvlu-blip/AI-practical-.git